Beyond Saving.
Toward Financial Intelligence.

A community-finance prototype for Rwanda.

01 / The problem and our approach

The problem we are solving

Community finance involves more than collecting contributions. Members need to understand their savings and obligations. Group leaders need reliable records, clear approval steps and visibility into available cash. Cooperatives need to connect daily operations with member value. Across these activities, people need records they can understand and access that respects member privacy.

When these activities are managed through disconnected records, it becomes harder to answer practical questions: What has been paid? What is still owed? How much can safely be lent? Who approved an expense? What information may be shared?

HURIZA brings these workflows into one platform, with separate workspaces and server-controlled access. It is designed to make records easier to follow and financial decisions easier to explain.

How HURIZA responds

- **One connected view:** personal finance, savings groups and cooperative management.
- **Traceable activity:** balanced journals, approval workflows, simulated payment outcomes and reversal-based corrections.
- **Understandable decisions:** Guardian explains deterministic calculations; scenario tools let users explore assumptions.
- **Context with attribution:** public Insights uses curated, published NISR aggregates with source and methodology notes.

Watch the product in action

Video walkthrough: https://youtu.be/8kvInbFDPYE

Use the walkthrough for a guided introduction, then follow the local setup below to explore the project yourself.

Prototype scope

All financial transactions, MoMo/payment outcomes and USSD operations are simulated. No real money is held or transferred. Published national statistics provide context; synthetic platform records do not demonstrate adoption, programme outcomes or real-world impact.

02 / What you can explore

Personal finance

Track personal financial records and savings goals, review your own group contributions and loans, and explore a savings planner. Public email registration requires verification and starts with an empty personal account. Organization access follows server-managed membership.

Savings groups: Itsinda and Ikimina

Itsinda (accumulating savings) supports contributions, loan eligibility, approval, reserves, repayment schedules and cycle settlement. **Ikimina (rotating savings)** supports member obligations, recipient order, rounds and simulated payouts. Group setup, invitations and administration are implemented, with access and workflow constraints enforced by the server.

Cooperative management

Manage cooperative profiles, member records and share capital, production deliveries, sales, staged expenses, governance events and surplus allocations. Investment scenarios explore potential choices without executing an investment or transferring funds.

Guardian and scenario planning

Guardian provides rule-based guidance grounded in deterministic calculations and authorized records. It explains results without approving loans or executing transactions. The current explanation layer uses templates; no external LLM provider is connected or required. Hypothetical scenarios do not change the ledger.

Insights and accessible interactions

Public Insights and methodology pages explain published evidence separately from operational data. Authorized CSV exports support personal and group record review. A browser USSD simulator demonstrates an interaction pattern without contacting a telecom network.

Mobile and offline support

The repository includes a web app manifest, service worker and IndexedDB-backed queue for supported actions. Queued work requires server authorization when synchronized; full offline feature parity is not claimed.

03 / Technology and architecture

Technology stack

Layer	Technology	Role in HURIZA
Runtime	Node.js 24, npm	Runtime and dependency management
Application	Next.js 16 App Router, React 19	Public pages, workspaces and server routes
Language	TypeScript 5.9	Shared contracts and typed code
Interface	CSS Modules, shared CSS, Tailwind CSS 4, Lucide React	Styling toolchain and icons
Database	PostgreSQL, PGlite, node-postgres (pg)	SQL transactions and database access
Identity	Firebase Authentication, Firebase Admin SDK	Email identity, verification and sessions
Validation	Zod 4	Request validation
Reporting	Google Sheets API, google-auth-library	Optional spreadsheet integration
Email	Resend	Optional contact-form delivery
Verification	Node test runner, tsx, TypeScript compiler	Financial/security tests and static checks

Drizzle ORM is installed, but financial services currently use parameterized SQL through the project's database interface. Its presence does not imply ORM-based implementation throughout the app.

Request and data flow

```
Browser / React workspace
 -> Next.js server routes
 -> Verified session + account + tenant + membership
 -> Authorized financial or reporting service
 -> PostgreSQL transactions and versioned SQL migrations
 -> Scoped response to the browser
```

HURIZA follows a clean layered architecture connecting user interactions, workspace domain logic, and persistent state safely.

The app is a modular monolith. Firebase handles email-based identity; a separate phone/PIN path supports provisioned member accounts. The server derives roles and memberships rather than accepting authority from the browser.

Financial integrity by design

Money uses safe integer RWF, with BigInt intermediate arithmetic. Posted journals are immutable and corrected through reversals. Payment workflows separate approval from simulated provider success before posting, with idempotency checks for repeat processing. Financial queries must be scoped by demo tenant and organization; personal records are additionally scoped to their owner.

04 / Run HURIZA locally

1. Install dependencies

Use **Node.js 24 and npm**. Open a terminal in the repository directory. Commands below use PowerShell on Windows; on macOS/Linux, replace Copy-Item with cp.

```
npm ci --legacy-peer-deps
```

For a fresh checkout, create the local configuration file from .env.example. Keep .env.local private: it is already ignored by Git and must not be committed or included in a submission. Share .env.example and the safe emulator settings below instead. Preserve any existing .env.local; use a separate checkout rather than overwriting working credentials.

```
Copy-Item .env.example .env.local
```

2. Configure an isolated local demo

Set these values in the new .env.local. Keep optional integrations unset. These identifiers target the local Authentication Emulator, not a live Firebase project.

```
DATABASE_URL=
NEXT_PUBLIC_FIREBASE_API_KEY=demo-local NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN=localhost
NEXT_PUBLIC_FIREBASE_PROJECT_ID=demo-huriza
NEXT_PUBLIC_FIREBASE_APP_ID=1:1234567890:web:huriza-local FIREBASE_PROJECT_ID=demo-huriza
FIREBASE_AUTH_EMULATOR_HOST=127.0.0.1:9099
NEXT_PUBLIC_FIREBASE_AUTH_EMULATOR_URL=http://127.0.0.1:9099
COOKIE_SECURE=false HURIZA_SHEETS_PRIMARY=false HURIZA_SHEETS_AUTO_SYNC=false RESEND_API_KEY=
```

3. Start authentication, then seed and run

In **terminal A**, start the emulator and leave it running:

Shell
```
npx firebase-tools emulators:start --only auth
```

In **terminal B**, wait for the emulator to be ready, then run:

Shell
```
npm run seed:demo
npm run dev
```

Open http://127.0.0.1:3000. Read the five independent fixture accounts from ignored .local/firebase-test-accounts.json.

Sign out and sign in as another fixture to explore a different role. New signups start empty.

PGlite has one process owner. Finish seeding before starting Next.js. Do not point another app or administrative process at the same .local/postgres directory. HURIZA_DB_PATH selects a separate directory for isolated work. The seeder refuses a non-demo Firebase project./ Configuration and verification

Optional integrations

Integration	Configuration and behavior
External PostgreSQL	Set DATABASE_URL. Use external PostgreSQL for multi-process operation; PGlite is embedded PostgreSQL for one local process.
Live Firebase	Supply owner-approved browser configuration and server-only Admin credentials or Application Default Credentials. Remove emulator settings; use secure cookies over HTTPS.
Google Sheets exports	Configure GOOGLE_SHEETS_SPREADSHEET_ID, service-account access and organization binding. Enable HURIZA_SHEETS_AUTO_SYNC only with a running worker.
Sheets-primary demo	HURIZA_SHEETS_PRIMARY uses an encrypted master snapshot with in-memory PGlite. Requires explicit bootstrap, a separate encryption key and one application writer.
Resend	Set RESEND_API_KEY, CONTACT_TO_EMAIL and CONTACT_FROM_EMAIL. Without an API key, the contact endpoint reports simulation mode.

Spreadsheet viewers can read copied records independently of workspace permissions. Review sharing before enabling exports. Sheets-primary is a bounded hackathon/demo mode with single-writer and recovery constraints.

Keep .env.local, service-account credentials, encryption keys and fixture credentials out of version control. NEXT_PUBLIC_* values reach the browser and must never contain server secrets.

Development checks

```
npm run typecheck
npm test
npm run build
```

Run automated tests and checks locally using npm test, npm run typecheck, and npm run lint prior to submitting or pushing changes.

Tests cover financial rules, authorization, privacy, savings groups, Guardian and Sheets behavior. These are commands to run against your checkout, not a claim that this README update performed or passed application checks.

For emulator-backed HTTP checks, keep the local emulator and app running with fixtures seeded, then run **npm run verify:auth** in another terminal. See testing prerequisites. Never direct the seeder or verifier at a live Firebase project.

For a production-mode local preview, build first and use **npm start** with matching database and authentication configuration. A development emulator .env.local is not a production configuration. Publication or deployment requires separate founder authorization.

05 / Project guide and responsible use

Repository map

src/app/	Pages, layouts and HTTP API routes
src/components/	Public interface and workspace components
src/lib/	Contracts, Guardian rules and offline queue
src/server/	Auth, permissions, finance and Sheets
migrations/	Versioned PostgreSQL schema changes
scripts/	Setup, provisioning and verification
tests/	Financial, security and integration tests
data/	Published evidence and validation pipeline
docs/	Contracts, methodology and operations
public/	Brand assets, manifest and service worker
output/pdf/	Shareable PDF edition of this README

The repository is structured with source code under /src (including app routes, components, and server logic), core domain services under /lib, and test suites under /tests.

Useful starting points

Public routes include /, /demo, /insights and /methodology. Authentication starts at /login or /signup. Project workspaces include /app/personal, /app/groups, /app/cooperative, /app/guardian and /app/simulator, with access enforced by the server.

Further reading

- Architecture and financial model
- Account contracts and implementation contracts
- Security and testing
- Google Sheets and Sheets-primary demo
- Data sources, methodology and AI disclosure

Some supporting documents retain historical status notes. For implemented behavior, consult the current source, migrations and tests; this guide does not assert a current deployment or certification status. In the PDF, documentation paths refer to files in the repository.

Evidence, limitations and ownership

Public evidence is curated from published NISR FinScope 2024 and EICV7 material. No restricted microdata is analyzed; national aggregates are not joined with member records to claim impact. References and caveats appear in the public methodology and data documentation.

HURIZA remains a prototype. Live payment and telecom connections, production operational hardening, hosted verification and regulated financial-service approval are not claimed. Phone/PIN access and offline workflows need review for the intended operating environment. Kinyarwanda text requires reviewed localization.

Development used AI assistance under human direction. Dependencies retain their own licenses; published evidence

remains attributed to its publishers. No open-source license is asserted for this repository. Ownership and competition IP terms require clarification before commercial rights are claimed.

Explore the walkthrough click that link :huriza walkthrough